DNS Rebinding Attack

Phase 1: The Setup (Building the Trap)

- **Step 1: Buy a Domain**
  - Spend $10 to buy a domain name like `attacker.com`.
- **Step 2: Build a Fake DNS Server**
  - Write a custom script for a malicious DNS server.
  - Program it to change answers dynamically based on request count.
  - Set rule: First query returns public IP. Second query returns `127.0.0.1`.
- **Step 3: Build a Public Web Server**
  - Launch a standard web server at public IP `x.x.x.x`.
  - Upload a webpage containing a data-stealing JavaScript payload.

Phase 2: The Attack (Executing the Trick)

- **Step 4: Lure the Victim**
  - Host a fake game or free video stream.
  - Trick the victim into visiting `http://attacker.com`.
- **Step 5: Tell the First Lie (The Bait)**
  - Wait for the victim's initial DNS lookup request.
  - Respond with your public server IP (`x.x.x.x`).
  - Inject a `TTL = 0` flag into the DNS response header.
  - Force the browser to cache nothing.
- **Step 6: Deliver the Payload**
  - Accept the victim's browser connection at `x.x.x.x`.
  - Serve the webpage and execute the malicious JavaScript file.
- **Step 7: Wait for the Reset**
  - Pause the background script for a few seconds.
  - Allow the browser to naturally purge the expired IP address.
- **Step 8: Trigger the Re-lookup**
  - Execute a background `fetch()` command to `http://attacker.com`.
  - Force the browser to send a second DNS lookup query.
- **Step 9: Tell the Second Lie (The Switch)**
  - Intercept the second lookup on your custom DNS server.
  - Switch the target address completely.
  - Return the local machine loopback address: `127.0.0.1`.
- **Step 10: Bypass the Same-Origin Policy**
  - Let the browser evaluate the request origin text.
  - Exploit the fact that the domain name string remains `attacker.com`.
  - Watch the browser approve access to the local port.
- **Step 11: Collect the Stolen Loot**
  - Use JavaScript to harvest internal data from local services.
  - Exfiltrate the stolen loot back to your permanent collection server.

Summary: DNS Rebinding Attack

DNS Rebinding is an architectural exploit that bypasses the browser's Same-Origin Policy (SOP) by exploiting the gap between domain names (used for browser security) and IP addresses (used for network routing).

1. **The Infrastructure**: The attacker configures a malicious DNS nameserver for a domain they control (e.g., `attacker.com`) and programs it to alternate its IP responses on demand.
2. **The Initial Bait**: When the victim visits the site, the DNS server returns the attacker's public web server IP with a `TTL = 0` (Time-to-Live), ensuring the browser never saves the IP in its cache.
3. **The Payload Delivery**: The browser connects to the public server and executes a malicious JavaScript payload.
4. **The Switch**: The JavaScript triggers a background request to the exact same domain name. Because the cache expired instantly, the browser forces a second DNS lookup.
5. **The Local Breach**: The attacker's DNS server now switches the answer, returning a local loopback address like `127.0.0.1`.
6. **The Security Failure**: The browser's SOP approves the request because the domain text (`attacker.com`) matches the script's origin, completely blind to the fact that the underlying traffic is now being routed straight into internal machine services.
7. **Exfiltration**: The script interacts with the local target, harvests private data, and exfiltrates it back to the attacker's server.

How the Request Flows

